I solver MIP: che cosa fanno

Indice

- Dall'algoritmo branch-and-cut a un solver MIP
  - Dizionario tra gli appunti e il solver
- I componenti di un solver MIP
  - Presolve
  - Piani di taglio
  - Euristiche primali
  - Selezione della variabile e del nodo di branching
  - Tolleranze e gap
- Come si legge il log di Gurobi
  - Un problema del catalogo risolto alla radice
  - Un MIP generico
  - La stessa istanza senza presolve, tagli ed euristiche
  - Il gap come criterio di arresto
  - Selezione della variabile di branching
- Errori tipici
- In sintesi

Obiettivi

Alla fine di questo capitolo sarai in grado di:

- nominare i componenti di un solver MIP e dire su quale lato dei bound agisce ciascuno di essi;
- spiegare presolve, tagli, euristiche primali, regole di branching, tolleranze e gap;
- leggere il log di Gurobi e seguire i due bound, la soluzione incumbent e l'albero;
- spiegare perché un solver può fermarsi con una soluzione che non è ottima.

Prerequisiti

- Da questo modulo, i capitoli «Branch-and-bound basato sulla programmazione lineare» e «Algoritmi branch-and-cut»: le operazioni di branching, bounding e pruning, i tagli e la soluzione incumbent
- Da questo modulo, il capitolo «Bound primali e duali»: i lower bound e gli upper bound e il gap di ottimalità
- Per imparare a usare Gurobi nella pratica, vedi il modulo «Modellazione MIP» (fabiofurini.github.io/modellazione-mip), capitolo 3 «Dal modello a Python/Gurobi»

Dall'algoritmo branch-and-cut a un solver MIP

Introduzione: Dall'algoritmo branch-and-cut a un solver MIP

- Nella pratica i problemi MIP si risolvono con un **solver MIP**: Gurobi, CPLEX, Xpress, HiGHS, SCIP
- Vediamo che il suo nucleo è l'algoritmo branch-and-cut, e quali componenti si aggiungono intorno a esso
- Ciascun componente agisce su uno dei due bound su $z(\mathrm{ILP})$: è la chiave per capire che cosa sta facendo il solver
- Chiudiamo con un dizionario tra la notazione degli appunti e le colonne del log di Gurobi

Definizione 1: Solver MIP

*Un **solver MIP** (Gurobi, CPLEX, Xpress, HiGHS, SCIP) è un programma che calcola soluzioni ottime di problemi MIP. Il suo nucleo è l'algoritmo branch-and-cut del capitolo precedente.*

I due lati dei bound

- Intorno al nucleo, il solver aggiunge diversi componenti
- Ciascuno di essi agisce su un lato dei bound su $z(\mathrm{ILP})$, qui per un problema di massimizzazione:

$$\underbrace{\mathrm{LB}}_{\text{lato primale}} \leq z(\mathrm{ILP}) \leq \underbrace{\mathrm{UB}}_{\text{lato duale}}$$

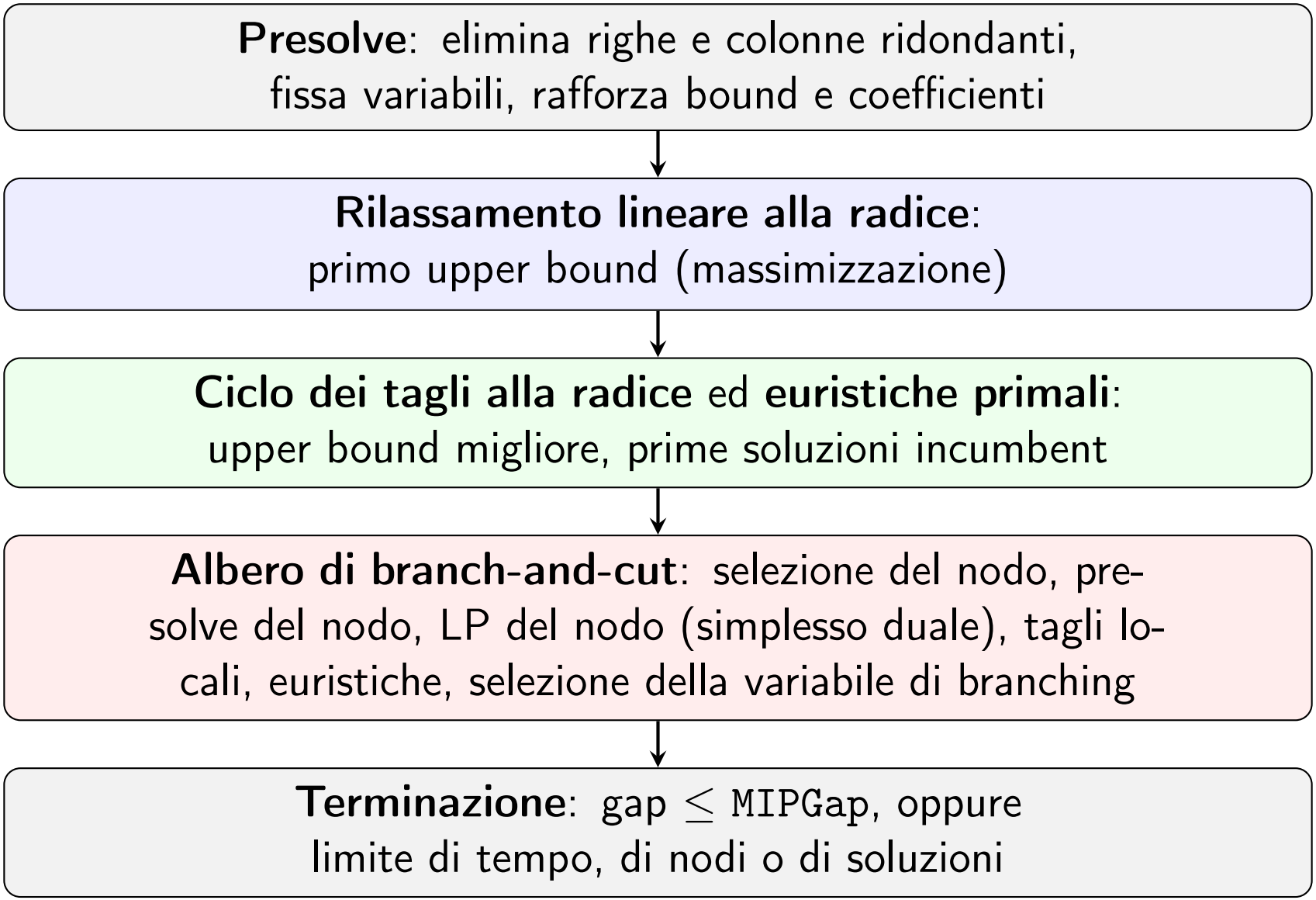
- I componenti sono gli stessi in tutti i solver
- I dettagli (quali tagli, quali euristiche, quali regole) cambiano da solver a solver e da versione a versione, e sono documentati solo in parte

Che cosa fa un solver MIP

Un solver MIP esegue il **presolve**,
poi il **nodo radice**
(rilassamento lineare, tagli, euristiche primali),
poi l'**albero di branch-and-cut**,

e si ferma quando il gap tra i bound
scende sotto una tolleranza o quando si raggiunge
un limite di tempo, di nodi o di soluzioni.

Le fasi di un solver MIP



Il solver parla una lingua sua:
prima di leggere un log, traduciamo
la notazione dei capitoli sul B&B e sul B&C
nei nomi delle colonne del log di Gurobi.

Dizionario tra gli appunti e il solver

Negli appunti (B&B, B&C)	Nel log di Gurobi
lower bound LB (max), soluzione incumbent $\bar{\boldsymbol{x}}$	`Incumbent`
upper bound UB (max)	`BestBd` (best bound)
gap di ottimalità	`Gap`
lista L dei problemi LP da elaborare	`Unexpl` (nodi non esplorati)
problemi LP già elaborati	`Expl` (nodi esplorati)
$z(\mathrm{LP}_k)$ del nodo di branching corrente	`Obj`
livello del nodo nell'albero	`Depth`
variabili intere frazionarie in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	`IntInf`

- In un problema di minimizzazione i ruoli si scambiano: `Incumbent` è un upper bound e `BestBd` è un lower bound

Riepilogo

- Un solver MIP è un programma costruito intorno all'algoritmo **branch-and-cut**
- Presolve, nodo radice (rilassamento lineare, tagli, euristiche), albero, terminazione: ogni componente agisce sul **lato primale** (LB) o sul **lato duale** (UB) dei bound
- Nel log di Gurobi, `Incumbent` e `BestBd` sono i due bound (massimizzazione), `Gap` il gap di ottimalità; in minimizzazione i ruoli si scambiano

I componenti di un solver MIP

Introduzione: I componenti di un solver MIP

- Guardiamo dentro il solver, un componente alla volta: presolve, piani di taglio, euristiche primali, regole di branching, tolleranze e gap
- Per ciascuno diciamo che cosa fa e su quale bound agisce
- Sono gli ingredienti che riconosceremo, riga per riga, nei log di Gurobi

Definizione 2: presolve

*Il **presolve** trasforma un problema MIP in uno equivalente: stesso valore ottimo, e una soluzione ottima del problema originale si ricava da una soluzione ottima del nuovo problema.*

Le operazioni del presolve

- Il nuovo problema è più piccolo e, spesso, ha un rilassamento lineare più forte

- Le operazioni tipiche sono tre:

1. eliminare i **vincoli ridondanti** e **fissare le variabili** il cui valore è implicato dagli altri vincoli;
2. **rafforzamento dei bound** (bound tightening): ricavare upper bound più piccoli e lower bound più grandi sulle variabili;
3. **rafforzamento dei coefficienti** (coefficient tightening): modificare i coefficienti di un vincolo in modo che le soluzioni intere restino le stesse, mentre il rilassamento lineare diventa più piccolo.

Un piccolo esempio mostra all'opera
il rafforzamento dei coefficienti:
stesse soluzioni intere, stessa dimensione del modello,
ma un upper bound migliore.

Esempio (1/3)

- Consideriamo il problema con tre variabili binarie

$$\begin{array}{lrcl} \max & x_1 + x_2 + x_3 & & \\ \text{soggetto a} & 6\,x_1 + 6\,x_2 + 3\,x_3 & \leq & 10 \\ & x_1, \quad x_2, \quad x_3 & \in & \{0,1\} \end{array}$$

- Il vincolo è soddisfatto dagli stessi sei punti binari del vincolo

$$7\,x_1 + 7\,x_2 + 3\,x_3 \leq 10$$

Esempio (2/3)

- Infatti $x_1 = x_2 = 1$ viola entrambi i vincoli:

$$12 > 10 \qquad \text{e} \qquad 14 > 10$$

- Ogni altro punto li soddisfa entrambi: i primi membri più grandi sono

$$9 \leq 10 \qquad \text{e} \qquad 10 \leq 10$$

Esempio (3/3)

- Il rilassamento lineare del primo vincolo ha valore ottimo $z(\mathrm{LP}) = \frac{13}{6}$, raggiunto per esempio in

$$\tilde{\boldsymbol{x}} = \left(1, \tfrac{1}{6}, 1\right)$$

è frazionaria, e $x_1 + x_2 = \frac{7}{6}$

- Il rilassamento lineare del secondo vincolo ha valore ottimo $2 = z(\mathrm{ILP})$

- L'upper bound scende da $\frac{13}{6}$ a 2 senza cambiare le soluzioni intere né la dimensione del modello

Presolve del nodo

- Lo stesso ragionamento si può applicare in ogni nodo di branching: **presolve del nodo**, o **propagazione dei domini**
- Un vincolo di branching $x_j \leq 0$ può implicare che altre variabili si possano fissare

Piani di taglio

- Al nodo radice il solver esegue diversi **round di tagli**; alla fine, il log riporta quanti tagli di ciascuna famiglia sono stati aggiunti

- Le famiglie che compaiono nei nostri log sono:
  - `Gomory` (tagli di Gomory)
  - `MIR` (tagli di mixed-integer rounding, che generalizzano i tagli di Gomory ai problemi MIP)
  - `StrongCG` (tagli di Chvátal–Gomory)
  - `Lift-and-project`

- I tagli si generano anche nell'albero, meno spesso che alla radice (branch-and-cut)

Euristiche primali

- Le **euristiche primali** cercano buone soluzioni intere ammissibili: forniscono la soluzione incumbent e il lower bound LB (massimizzazione)
- Esempi sono l'arrotondamento della soluzione del rilassamento lineare, il **diving** e la ricerca locale intorno alla soluzione incumbent (capitolo «Euristiche MIP»)

Nuove soluzioni incumbent nel log

- Una riga che inizia con `H` riporta una nuova soluzione incumbent trovata da un'euristica
- Una riga che inizia con `*` ne riporta una trovata come soluzione ottima intera del problema LP di un nodo (potatura per integralità)
- La riga `Found heuristic solution` riporta una soluzione trovata prima del problema LP della radice

Selezione della variabile di branching

- Le strategie di selezione della variabile di branching del capitolo sul B&B (massima inammissibilità, strong branching, pseudocosti) sono tutte disponibili in un solver
- Lo strong branching dà alberi piccoli ma è costoso; i pseudocosti costano poco ma all'inizio sono poco affidabili
- I solver le combinano nel **reliability branching**: strong branching su una variabile finché i suoi pseudocosti non sono stati osservati un numero sufficiente di volte

Il branching in Gurobi

- In Gurobi il parametro `VarBranch` sceglie la strategia:
  - -1 automatica (predefinita)
  - 0 pseudo costo ridotto
  - 1 pseudo prezzo ombra
  - 2 massima inammissibilità
  - 3 strong branching

- La selezione del nodo combina **best bound** (si sceglie il nodo con il valore $z(\mathrm{LP}_k)$ più grande, il che fa scendere UB) e **ricerca in profondità** (che trova presto soluzioni intere)

Aritmetica a virgola mobile

- Un solver MIP lavora in **aritmetica a virgola mobile**
- Un valore è intero, un vincolo è soddisfatto, un costo ridotto è non positivo, a meno di una **tolleranza**
- I valori predefiniti in Gurobi 13 sono nella tabella della slide successiva

Le tolleranze in Gurobi 13

Parametro	Predefinito	Significato
`IntFeasTol`	10^{-5}	una variabile è intera se dista meno di 10^{-5} da un intero
`FeasibilityTol`	10^{-6}	violazione massima di un vincolo
`OptimalityTol`	10^{-6}	tolleranza sui costi ridotti (ottimalità LP)
`MIPGap`	10^{-4}	gap relativo a cui il solver si ferma
`MIPGapAbs`	10^{-10}	gap assoluto a cui il solver si ferma

Che cosa significa «Optimal solution found»

"`Optimal solution found (tolerance 1.00e-04)`"
significa che la soluzione incumbent è ottima
a meno di un gap relativo di 10^{-4}.

Con un `MIPGap` più grande il solver si ferma
con una soluzione che può **non** essere ottima,
e scrive la stessa frase.

Riepilogo (1/2)

Presolve e tagli:

- Il presolve dà un problema equivalente, più piccolo e spesso con un rilassamento lineare più forte; nell'esempio il rafforzamento dei coefficienti abbassa il bound da $\frac{13}{6}$ a 2
- Round di tagli alla radice, meno tagli nell'albero

Euristiche primali:

- Forniscono la soluzione incumbent e LB (massimizzazione): righe `H` nel log; le righe $*$ vengono da soluzioni intere dei problemi LP dei nodi

Riepilogo (2/2)

Branching:

- Il reliability branching combina strong branching e pseudocosti; in Gurobi `VarBranch` sceglie la strategia
- La selezione del nodo combina best bound e ricerca in profondità

Tolleranze e gap:

- Integralità, ammissibilità e ottimalità valgono a meno di una tolleranza
- "`Optimal solution found`" significa ottima a meno di `MIPGap`

Come si legge il log di Gurobi

Introduzione: Come si legge il log di Gurobi

- Il log è l'unica finestra su quello che fa il solver: impariamo a leggerlo riga per riga
- Partiamo da un piccolo problema del catalogo risolto alla radice, poi un MIP generico G_1 con 30 variabili intere
- Disattiviamo presolve, tagli ed euristiche, cambiamo il gap e la regola di branching, e vediamo che cosa cambia nel log

I log di questo capitolo

- Tutti i log sono prodotti da Gurobi 13.0.3 (`gurobipy`) con `Threads` $= 1$ e `Seed` $= 0$, così che l'esecuzione sia deterministica
- Con altre versioni o altri computer il numero di nodi e di iterazioni può cambiare
- I numeri a sinistra sono numeri di riga aggiunti per i commenti; le righe `Set parameter ...` sono omesse

Un problema del catalogo risolto alla radice

- Partiamo dal problema $P41$ del catalogo dei problemi MIP: tre variabili intere non negative e quattro vincoli

$$
\begin{array}{lrcrcrcl}
\max & 2\,x_1 & + & x_2 & + & 3\,x_3 & & \\
\text{soggetto a} & x_1 & + & x_2 & + & x_3 & \leq & 4 \\
 & & & x_2 & & & \leq & 2 \\
 & & & & & x_3 & \leq & 3 \\
 & 3\,x_1 & & & + & x_3 & \leq & 6 \\
 & x_1, & & x_2, & & x_3 & \in & \mathbb{Z}_{\geq 0}
\end{array}
$$

- Lo risolviamo con i parametri predefiniti e leggiamo il log riga per riga

Log di Gurobi: problema $P41$, parametri predefiniti (1/5)

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| LogToConsole  0
08| Threads  1
09|
```

- **Righe 01–08**: versione di Gurobi, computer, thread usati (1, poiché `Threads` $= 1$); parametri con valori non predefiniti (`Seed` $= 0$ non compare, perché 0 è il suo valore predefinito)

Log di Gurobi: problema $P41$, parametri predefiniti (2/5)

```
10| Optimize a model with 4 rows, 3 columns and 7 nonzeros (Max)
11| Model fingerprint: 0xedc792ce
12| Model has 3 linear objective coefficients
13| Variable types: 0 continuous, 3 integer (0 binary)
14| Coefficient statistics:
15|   Matrix range     [1e+00, 3e+00]
16|   Objective range  [1e+00, 3e+00]
17|   Bounds range     [0e+00, 0e+00]
18|   RHS range        [2e+00, 6e+00]
19|
```

- **Righe 10–13**: dimensione del modello: 4 righe, 3 colonne, 7 elementi non nulli, `(Max)`; i tipi delle variabili (3 intere, nessuna binaria)
- **Righe 14–18**: **statistiche dei coefficienti** (slide successiva)

Statistiche dei coefficienti

- Le **righe 14–18** del log riportano le **statistiche dei coefficienti**: il più piccolo e il più grande valore assoluto dei coefficienti
- Intervalli che coprono molti ordini di grandezza segnalano difficoltà numeriche
- Qui tutti i valori sono compresi tra 1 e 6

Log di Gurobi: problema $P41$, parametri predefiniti (3/5)

```
20| Found heuristic solution: objective 6.0000000
21| Presolve removed 2 rows and 0 columns
22| Presolve time: 0.00s
23| Presolved: 2 rows, 3 columns, 5 nonzeros
24| Variable types: 0 continuous, 3 integer (0 binary)
25|
26| Root relaxation: objective 1.100000e+01, 1 iterations, 0.00 seconds (0.00 work units)
```

- **Riga 20**: un'euristica trova una soluzione di valore 6 prima del presolve, quindi $\mathrm{LB} = 6$
- **Righe 21–23**: il presolve elimina 2 righe: al modello restano 2 righe e 3 colonne
- **Riga 26**: **rilassamento alla radice**: il valore è 11 e basta 1 iterazione del simplesso

Log di Gurobi: problema $P41$, parametri predefiniti (4/5)

```
27|
28|     Nodes    |    Current Node    |     Objective Bounds      |     Work
29|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
30|
31| *    0     0               0      11.0000000   11.00000  0.00%     -    0s
32|
```

- **Riga 31**: la soluzione del problema LP della radice è intera, quindi il nodo è risolto subito (una riga che inizia con $*$, come visto con le euristiche primali)
- $\mathrm{LB} = \mathrm{UB} = 11$, e la soluzione è ottima

Log di Gurobi: problema $P41$, parametri predefiniti (5/5)

```
33| Explored 1 nodes (1 simplex iterations) in 0.00 seconds (0.00 work units)
34| Thread count was 1 (of 20 available processors)
35|
36| Solution count 2: 11 6
37|
38| Optimal solution found (tolerance 1.00e-04)
39| Best objective 1.100000000000e+01, best bound 1.100000000000e+01, gap 0.0000%
```

- **Righe 33–39**: 1 nodo esplorato e 1 iterazione del simplesso; 2 soluzioni trovate, 11 e 6; la soluzione di valore 11 è ottima (tolleranza 10^{-4})

$P41$ si risolve alla radice, senza albero.
Per vedere all'opera tutti i componenti
serve un'istanza più grande.

Un MIP generico

- D'ora in poi usiamo un **MIP generico con** 30 **variabili intere**, che chiamiamo G_1
- È un problema PLI puro con 10 vincoli, nella forma

$$\max \quad \boldsymbol{c}'\boldsymbol{x}$$

$$\text{soggetto a} \quad A\boldsymbol{x} \leq \boldsymbol{b},$$

$$x_j \in \{0,1,2,3\}, \quad \forall j \in \{1,2,\ldots,30\}.$$

I dati di G_1

- Tutti i dati sono interi e pseudo-casuali (Python `random.Random(1)`):
  - a_{ij} uniforme in $\{1, \ldots, 20\}$
  - c_j uniforme in $\{5, \ldots, 25\}$
  - $b_i = \sum_j a_{ij}$

- Lo script `script/genera_log.py` di questo capitolo costruisce l'istanza e produce tutti i log

Log di Gurobi: MIP generico G_1, parametri predefiniti (1/11)

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| LogToConsole  0
08| Threads  1
09|
```

- **Righe 01–08**: la stessa intestazione di $P41$: versione, computer, thread, parametri non predefiniti

Log di Gurobi: MIP generico G_1, parametri predefiniti (2/11)

```
10| Optimize a model with 10 rows, 30 columns and 300 nonzeros (Max)
11| Model fingerprint: 0xb023d504
12| Model has 30 linear objective coefficients
13| Variable types: 0 continuous, 30 integer (0 binary)
14| Coefficient statistics:
15|   Matrix range     [1e+00, 2e+01]
16|   Objective range  [5e+00, 2e+01]
17|   Bounds range     [3e+00, 3e+00]
18|   RHS range        [3e+02, 4e+02]
19|
```

- **Righe 10–13**: 10 righe, 30 colonne, 300 elementi non nulli; `(Max)`: massimizzazione; le variabili sono intere ma non binarie

Log di Gurobi: MIP generico G_1, parametri predefiniti (3/11)

```
20| Found heuristic solution: objective 429.0000000
21| Presolve time: 0.00s
22| Presolved: 10 rows, 30 columns, 300 nonzeros
23| Variable types: 0 continuous, 30 integer (0 binary)
24|
25| Root relaxation: objective 5.991749e+02, 13 iterations, 0.00 seconds (0.00 work units)
26|
```

- **Riga 20**: un'euristica trova una soluzione di valore 429: $\mathrm{LB} = 429$
- **Righe 21–23**: il presolve non cambia la dimensione del modello
- **Riga 25**: **rilassamento alla radice**: il valore è 599.17487 (upper bound), dopo 13 iterazioni del simplesso

Log di Gurobi: MIP generico G_1, parametri predefiniti (4/11)

```
27|     Nodes    |    Current Node    |     Objective Bounds      |     Work
28|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
29|
30|     0     0  599.17487    0    6  429.00000  599.17487  39.7%     -    0s
```

- **Righe 27–28**: intestazione del **log dei nodi**: `Expl` e `Unexpl` sono i nodi esplorati e non esplorati; `Obj`, `Depth`, `IntInf` descrivono il nodo corrente; `Incumbent`, `BestBd`, `Gap` sono i due bound e il gap
- **Riga 30**: nodo radice: 6 variabili intere sono frazionarie, e il gap è 39.7%

Log di Gurobi: MIP generico G_1, parametri predefiniti (5/11)

```
27|     Nodes    |    Current Node    |     Objective Bounds      |     Work
28|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
31|  H    0     0                     563.0000000  599.17487  6.43%     -    0s
32|  H    0     0                     576.0000000  599.17487  4.02%     -    0s
33|  H    0     0                     584.0000000  599.17487  2.60%     -    0s
34|  H    0     0                     587.0000000  599.17487  2.07%     -    0s
35|  H    0     0                     588.0000000  599.17487  1.90%     -    0s
36|       0     0  597.88443    0    5  588.00000  597.88443  1.68%     -    0s
37|  H    0     0                     589.0000000  597.88443  1.51%     -    0s
38|  H    0     0                     592.0000000  597.88443  0.99%     -    0s
```

- **Righe 31–35**: righe che iniziano con `H`: le euristiche primali trovano soluzioni incumbent migliori, da 563 a 588, mentre il best bound resta 599.17487
- **Riga 36**: comincia il **ciclo dei tagli** (slide successive)

Log di Gurobi: MIP generico G_1, parametri predefiniti (6/11)

```
27|     Nodes    |    Current Node    |     Objective Bounds      |     Work
28|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
39|     0     0  597.70993    0    5  592.00000  597.70993  0.96%     -    0s
40|     0     0  597.67133    0    5  592.00000  597.67133  0.96%     -    0s
41|     0     0  597.65097    0    4  592.00000  597.65097  0.95%     -    0s
42|     0     0  597.64318    0    5  592.00000  597.64318  0.95%     -    0s
43|     0     0  597.64271    0    5  592.00000  597.64271  0.95%     -    0s
44|     0     0  597.56576    0    5  592.00000  597.56576  0.94%     -    0s
45| H   0     0                      593.0000000  597.56576  0.77%     -    0s
46|     0     0  597.55314    0    6  593.00000  597.55314  0.77%     -    0s
47|     0     0  597.52752    0    6  593.00000  597.52752  0.76%     -    0s
48|     0     0  597.51828    0    6  593.00000  597.51828  0.76%     -    0s
```

- **Righe 36–77**: il **ciclo dei tagli**: ogni riga con 0 nodi esplorati è un nuovo round di tagli, e il best bound scende
- Si trovano altre soluzioni incumbent, fino a 593 (riga 45)

Log di Gurobi: MIP generico G_1, parametri predefiniti (7/11)

	Nodes		Current Node			Objective Bounds			Work	
	Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
49	0	0	597.47194	0	7	593.00000	597.47194	0.75%	-	0s
50	0	0	597.21692	0	6	593.00000	597.21692	0.71%	-	0s
51	0	0	597.20185	0	6	593.00000	597.20185	0.71%	-	0s
52	0	0	597.19580	0	6	593.00000	597.19580	0.71%	-	0s
53	0	0	597.07620	0	6	593.00000	597.07620	0.69%	-	0s
54	0	0	597.07620	0	6	593.00000	597.07620	0.69%	-	0s
55	0	0	597.07620	0	6	593.00000	597.07620	0.69%	-	0s
56	0	0	597.07620	0	6	593.00000	597.07620	0.69%	-	0s
57	0	0	597.07620	0	6	593.00000	597.07620	0.69%	-	0s
58	0	0	597.07620	0	7	593.00000	597.07620	0.69%	-	0s

- Il ciclo dei tagli continua, round dopo round: la soluzione incumbent resta 593 e il best bound scende lentamente

Log di Gurobi: MIP generico G_1, parametri predefiniti (8/11)

27	Nodes		Current Node			Objective Bounds			Work	
28	Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
59	0	0	597.07620	0	7	593.00000	597.07620	0.69%	-	0s
60	0	0	597.07620	0	6	593.00000	597.07620	0.69%	-	0s
61	0	0	597.07620	0	7	593.00000	597.07620	0.69%	-	0s
62	0	0	597.07620	0	8	593.00000	597.07620	0.69%	-	0s
63	0	0	597.07074	0	7	593.00000	597.07074	0.69%	-	0s
64	0	0	597.02014	0	9	593.00000	597.02014	0.68%	-	0s
65	0	0	597.01888	0	9	593.00000	597.01888	0.68%	-	0s
66	0	0	597.01587	0	9	593.00000	597.01587	0.68%	-	0s
67	0	0	596.92506	0	8	593.00000	596.92506	0.66%	-	0s
68	0	0	596.85717	0	8	593.00000	596.85717	0.65%	-	0s

- **Righe 53–62**: il best bound resta 597.07620 per diversi round; dalla riga 63 riprende a scendere

Log di Gurobi: MIP generico G_1, parametri predefiniti (9/11)

27	Nodes		Current Node			Objective Bounds			Work	
28	Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
69	0	0	596.85610	0	8	593.00000	596.85610	0.65%	-	0s
70	0	0	596.85474	0	8	593.00000	596.85474	0.65%	-	0s
71	0	0	596.85333	0	8	593.00000	596.85333	0.65%	-	0s
72	0	0	596.76534	0	8	593.00000	596.76534	0.63%	-	0s
73	0	0	596.76242	0	8	593.00000	596.76242	0.63%	-	0s
74	0	0	596.75427	0	9	593.00000	596.75427	0.63%	-	0s
75	0	0	596.74106	0	8	593.00000	596.74106	0.63%	-	0s
76	0	0	596.73942	0	9	593.00000	596.73942	0.63%	-	0s
77	0	0	596.73942	0	9	593.00000	596.73942	0.63%	-	0s
78	0	2	596.64707	0	9	593.00000	596.64707	0.62%	-	0s

- Alla fine del ciclo dei tagli (riga 77) il best bound è sceso da 599.17487 a 596.73942
- **Riga 78**: sulla radice si esegue il branching: 2 nodi attendono nella lista, il gap è 0.62%

Log di Gurobi: MIP generico G_1, parametri predefiniti (10/11)

```
79|
80| Cutting planes:
81|   Gomory: 1
82|   Lift-and-project: 1
83|   MIR: 5
84|   StrongCG: 3
85|
```

- **Righe 81–84**: tagli aggiunti: 1 di Gomory, 1 lift-and-project, 5 MIR, 3 StrongCG

Log di Gurobi: MIP generico G_1, parametri predefiniti (11/11)

```
86| Explored 47 nodes (259 simplex iterations) in 0.06 seconds (0.04 work units)
87| Thread count was 1 (of 20 available processors)
88|
89| Solution count 9: 593 592 589 ... 429
90|
91| Optimal solution found (tolerance 1.00e-04)
92| Best objective 5.930000000000e+02, best bound 5.930000000000e+02, gap 0.0000%
```

- **Righe 86–92**: 47 nodi esplorati, 259 iterazioni del simplesso; 9 soluzioni trovate, la migliore è 593, ottima

La maggior parte del lavoro si fa alla radice

Qui la maggior parte del lavoro si fa alla radice:
le euristiche alzano il lower bound da 429 a 593
e i tagli abbassano l'upper bound da 599.17 a 596.65.

L'albero serve poi a **dimostrare**
che 593 è ottimo.

La stessa istanza senza presolve, tagli ed euristiche

- Quanto lavoro fanno risparmiare presolve, tagli ed euristiche? Li disattiviamo
- Con `Presolve` $= 0$, `Cuts` $= 0$ e `Heuristics` $= 0$ il solver è vicino all'algoritmo B&B basato su LP degli appunti, e vediamo l'albero

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0 (1/4)

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| Heuristics  0
08| Cuts  0
09| LogToConsole  0
10| Presolve  0
11| Threads  1
12|
```

- **Righe 06–11**: fra i parametri non predefiniti ora compaiono `Heuristics`, `Cuts` e `Presolve`

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0 (2/4)

```
13| Optimize a model with 10 rows, 30 columns and 300 nonzeros (Max)
14| Model fingerprint: 0xb023d504
15| Model has 30 linear objective coefficients
16| Variable types: 0 continuous, 30 integer (0 binary)
17| Coefficient statistics:
18|   Matrix range     [1e+00, 2e+01]
19|   Objective range  [5e+00, 2e+01]
20|   Bounds range     [3e+00, 3e+00]
21|   RHS range        [3e+02, 4e+02]
22|
23| Variable types: 0 continuous, 30 integer (0 binary)
24|
25| Root relaxation: objective 5.991749e+02, 13 iterations, 0.00 seconds (0.00 work units)
26|
```

- **Righe 22–23**: nessuna riga di euristica e nessuna riga di presolve: non c'è ancora una soluzione incumbent

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0 (3/4)

```
27|     Nodes    |    Current Node    |     Objective Bounds      |     Work
28|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
29|
30|     0     0  599.17487    0    6          -  599.17487      -     -    0s
31|     0     0  598.42295    0    6          -  598.42295      -     -    0s
32|     0     2  598.42295    0    6          -  598.42295      -     -    0s
33| *   33    22               31     584.0000000  597.70712  2.35%   1.6    0s
34| *   73    18               12     593.0000000  597.40703  0.74%   1.7    0s
```

- **Righe 30–32**: il bound della radice è 599.17487 e scende a 598.42295 prima del branching sulla radice; la colonna `Incumbent` contiene un trattino (-) perché non si conosce alcuna soluzione
- **Righe 33–34**: due soluzioni incumbent sono trovate come soluzioni ottime intere dei problemi LP dei nodi (*), ai nodi 33 e 73: valori 584 e 593

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0 (4/4)

```
35|
36| Explored 119 nodes (228 simplex iterations) in 0.00 seconds (0.00 work units)
37| Thread count was 1 (of 20 available processors)
38|
39| Solution count 2: 593 584
40|
41| Optimal solution found (tolerance 1.00e-04)
42| Best objective 5.930000000000e+02, best bound 5.930000000000e+02, gap 0.0000%
```

- **Riga 36**: per dimostrare che 593 è ottimo il solver esplora 119 nodi (228 iterazioni), contro i 47 nodi con i parametri predefiniti

Il gap come criterio di arresto

- Il solver si ferma quando il gap scende sotto `MIPGap`, che ha valore predefinito 10^{-4}
- Ripetiamo l'esecuzione precedente con `MIPGap` $= 0.01$, e vediamo che cosa scrive il solver alla fine

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0, MIPGap=0.01 (1/4)

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| MIPGap  0.01
08| Heuristics  0
09| Cuts  0
10| LogToConsole  0
11| Presolve  0
12| Threads  1
13|
```

- **Riga 07**: il nuovo parametro non predefinito MIPGap $= 0.01$

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0, MIPGap=0.01 (2/4)

```
14| Optimize a model with 10 rows, 30 columns and 300 nonzeros (Max)
15| Model fingerprint: 0xb023d504
16| Model has 30 linear objective coefficients
17| Variable types: 0 continuous, 30 integer (0 binary)
18| Coefficient statistics:
19|   Matrix range     [1e+00, 2e+01]
20|   Objective range  [5e+00, 2e+01]
21|   Bounds range     [3e+00, 3e+00]
22|   RHS range        [3e+02, 4e+02]
23|
24| Variable types: 0 continuous, 30 integer (0 binary)
25|
26| Root relaxation: objective 5.991749e+02, 13 iterations, 0.00 seconds (0.00 work units)
27|
```

- **Righe 14–26**: lo stesso modello e lo stesso rilassamento alla radice dell'esecuzione precedente

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0, MIPGap=0.01 (3/4)

```
28|     Nodes    |    Current Node    |     Objective Bounds      |     Work
29|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
30|
31|     0     0  599.17487    0    6          -  599.17487      -     -    0s
32|     0     0  598.42295    0    6          -  598.42295      -     -    0s
33|     0     2  598.42295    0    6          -  598.42295      -     -    0s
34| *   33    22               31     584.0000000  597.70712  2.35%   1.6    0s
35| *   73    18               12     593.0000000  597.40703  0.74%   1.7    0s
36|
```

- **Righe 34–35**: le stesse due soluzioni incumbent di prima, 584 e 593

Log di Gurobi: G_1, Presolve=0, Cuts=0, Heuristics=0, MIPGap=0.01 (4/4)

```
37| Explored 74 nodes (138 simplex iterations) in 0.00 seconds (0.00 work units)
38| Thread count was 1 (of 20 available processors)
39|
40| Solution count 2: 593 584
41|
42| Optimal solution found (tolerance 1.00e-02)
43| Best objective 5.930000000000e+02, best bound 5.970000000000e+02, gap 0.6745%
```

- **Riga 37**: il solver si ferma dopo 74 nodi invece di 119 (slide successiva)
- **Righe 42–43**: il solver scrive "`Optimal solution found (tolerance 1.00e-02)`" per il valore 593

Perché il solver si ferma dopo 74 nodi

- Con $\mathrm{LB} = 593$ il best bound è 597
- Il valore 597.40703 dell'ultima riga dei nodi è arrotondato per difetto dal solver, poiché tutti i dati sono interi
- Il gap è

$$\frac{597 - 593}{593} \approx 0.67\% < 1\%$$

Ottima a meno della tolleranza

- Il bound 597 certifica solo che 593 dista al più lo 0.67% dall'ottimo
- Qui la soluzione è in effetti ottima, ma il solver non può saperlo senza esplorare i 45 nodi rimanenti

Selezione della variabile di branching

- Con presolve, tagli ed euristiche disattivati, cambiamo solo `VarBranch`
- Usiamo G_1 e un secondo MIP generico G_2, con la stessa costruzione:
  - 30 variabili intere in $\{0, \ldots, 5\}$
  - 8 vincoli, con $b_i = \lfloor 5 \sum_j a_{ij}/3 \rfloor$
- La tabella della slide successiva riporta i nodi esplorati (tra parentesi le iterazioni del simplesso)

Nodi esplorati per ciascuna regola di branching

`VarBranch`	G_1	G_2
-1 automatica	119 (228)	566 (1144)
0 pseudo costo ridotto	119 (228)	566 (1144)
1 pseudo prezzo ombra	176 (313)	645 (1270)
2 massima inammissibilità	223 (392)	527 (1097)
3 strong branching	99 (184)	497 (934)

Che cosa dice la tabella

- Su queste due istanze le differenze sono moderate: lo strong branching dà gli alberi più piccoli, e la regola della massima inammissibilità dà il più grande su G_1 e il secondo più piccolo su G_2
- L'ordine delle regole cambia con l'istanza, quindi il comportamento di una regola è difficile da prevedere; la scelta automatica non è mai lontana dalla migliore
- Lo strong branching richiede meno nodi, ma ogni nodo costa di più: il solo numero di nodi non misura il tempo di calcolo

Leggere un log

Leggere un log significa seguire i due bound:

la colonna `Incumbent` è il **lato primale**
(euristiche e nodi interi),

la colonna `BestBd` il **lato duale**
(rilassamento, presolve, tagli, branching).

Riepilogo (1/2)

$P41$ e G_1 con i parametri predefiniti:

- $P41$ si risolve alla radice: la soluzione del problema LP della radice è intera, $\mathrm{LB} = \mathrm{UB} = 11$
- Su G_1 alla radice le euristiche alzano LB da 429 a 593 e i tagli abbassano UB da 599.17 a 596.65; l'albero dimostra l'ottimalità con 47 nodi

G_1 senza presolve, tagli ed euristiche:

- Nessuna soluzione incumbent fino al nodo 33; 119 nodi per dimostrare che 593 è ottimo

Riepilogo (2/2)

Il gap come criterio di arresto:

- Con `MIPGap` $= 0.01$ il solver si ferma dopo 74 nodi, con gap 0.67%, e scrive comunque "`Optimal solution found`"

Selezione della variabile di branching:

- Lo strong branching dà gli alberi più piccoli su G_1 e G_2, ma l'ordine delle regole cambia con l'istanza, e i nodi non misurano il tempo

Errori tipici

Introduzione: Errori tipici

- Elenchiamo gli errori più frequenti nella lettura dell'output di un solver MIP
- Riguardano il significato di "`Optimal solution found`", i bound in minimizzazione, il ruolo dell'albero, il numero di nodi e le regole del solver
- Ciascuno si può verificare sui log che abbiamo appena letto

Errori tipici

Osservazione 1: Errori tipici (1/3)

- ***Leggere "Optimal solution found" come una prova di ottimalità.*** *La frase riporta la tolleranza in uso: con* `MIPGap` $= 0.01$ *la soluzione può distare l'*1% *dall'ottimo.*
- ***Scambiare i bound in un problema di minimizzazione.*** *In un problema di minimizzazione* `Incumbent` *è l'upper bound e* `BestBd` *il lower bound.*

Errori tipici

Osservazione: Errori tipici (2/3)

- ***Credere che l'albero spieghi tutto.*** *Con i parametri predefiniti, la maggior parte del lavoro del solver avviene fuori dall'albero: nei nostri log il solo nodo radice risolve* $P41$ *e, su* G_1*, le euristiche e i tagli della radice danno la soluzione ottima e la maggior parte del bound.*

Errori tipici

Osservazione: Errori tipici (3/3)

- ***Confrontare il numero di nodi tra esecuzioni diverse.*** *I numeri dipendono dalla versione del solver, dal computer e dal seme; con altri parametri la stessa istanza può richiedere 47 o 223 nodi.*
- ***Aspettarsi che il solver segua le regole degli appunti.*** *Le scelte del solver (tagli, euristiche, selezione dei nodi) non sono documentate nel dettaglio, e cambiano con la versione.*

Riepilogo

Errori tipici:

- "`Optimal solution found`" vale solo a meno di `MIPGap`
- In minimizzazione `Incumbent` è l'upper bound e `BestBd` il lower bound
- Spesso la maggior parte del lavoro si fa alla radice, fuori dall'albero
- Il numero di nodi dipende da versione, computer, seme e parametri; le regole del solver non sono quelle degli appunti

In sintesi

Introduzione: In sintesi

- Raccogliamo le idee chiave del capitolo
- Chiudiamo con il cruscotto del MIP generico G_1: i due bound su $z(\mathrm{ILP})$ lungo il log con i parametri predefiniti

Riepilogo (1/3)

- Un solver MIP è un algoritmo branch-and-cut circondato da presolve, tagli, euristiche primali, regole di branching e di selezione dei nodi
- Presolve e tagli alzano il lower bound (minimizzazione), le euristiche trovano la soluzione incumbent, il branching chiude il gap
- Il log mostra i bound: `Incumbent` è il lato primale, `BestBd` il lato duale, `Gap` la loro distanza relativa

Riepilogo (2/3)

- Il solver si ferma quando il gap scende sotto `MIPGap`: la soluzione è ottima solo a meno di quella tolleranza
- Disattivare i componenti mostra quanto lavoro fa risparmiare ciascuno di essi su una data istanza

Riepilogo (3/3)

Cruscotto del MIP generico G_1**.** Lower bound e upper bound su $z(\mathrm{ILP}) = 593$ nel log con i parametri predefiniti di questo capitolo:

Fase	Lower bound	Upper bound
Soluzione euristica prima della radice	429	—
Rilassamento lineare alla radice	429	599
Nodo radice, euristiche e tagli	593	596
Fine dell'albero	593	593